7. Functional Abstraction

Describing a Machine by Its Function

A **functional abstraction** is a description of a machine in terms of its function, rather than its internal mechanism. Functional abstraction is common. We say that a given machine keeps food cold (without saying how it accomplishes this); or we say that a given machine is for driving (without saying if it is gas or electric).

Functional abstraction is also one of the basic analytical tools of cognitive science. We have already encountered one form of functional abstraction.

Logic gates themselves can be thought of as functional abstractions from concrete machines.

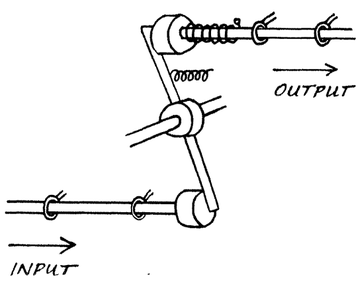
IN	OUT
0	1
1	0

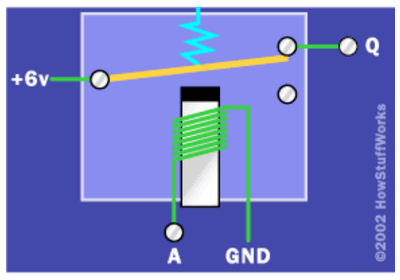
A NOT-gate.



Abstraction over internal description.

We describe a gate in terms of input-output profile, which is its function, rather than its internal machinery.

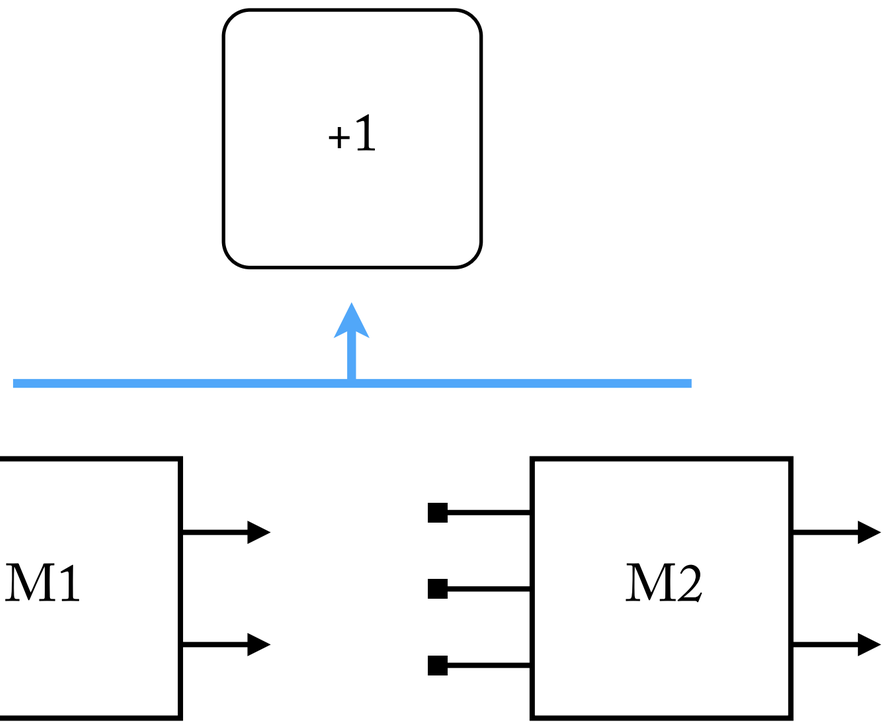




Physical implementations of a NOT-gate.

Functional abstraction recurs at many levels. So logic gates are functional abstractions. But we can also create functional abstractions for machines *made out of* logic gates.

Now that we have introduced the concept of computation, we can also describe an entire circuit in terms of the function it computes.



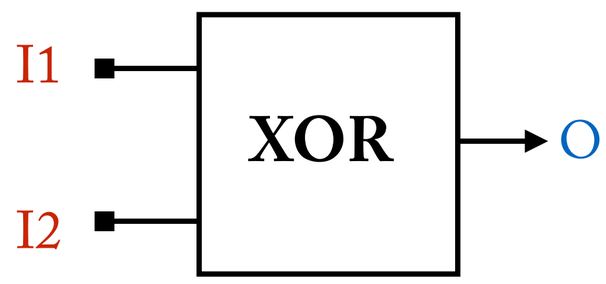
This is the kind of abstraction we engage in when we say that a calculator performs **multiplication**, or that an app on your phone **finds the fastest route home.**

These are all forms of functional abstraction in terms of the function computed.

Diagramming Abstraction

Concretely, we'll abstract over lower-level descriptions by drawing a "box" around a circuit, and then labeling the box with the function of that circuit. I'll call these **boxed** circuits.

For example, once you have shown how to build an XOR from NOT, AND, and OR-gates (HW1), then we can "box" that circuit. Then you can use XOR as a basic element in your other circuits:



In general, once you have defined a circuit for a given function (say, binary +1[0-7]), you can use that circuit later as part of other circuits. For example:

